Laboratorium 3

by Paweł Paduch (https://achilles.tu.kielce.pl/portal/author/332bd2d158a24964b2b98a7fc2879842) published 5 years ago, last modified 2 years ago

Delegaty, Eventy i Timery

Gdy korzystamy z VS2022 i wyższej należy zmienić we właściwościach projektu wersję .net core 3.1 na np. 6.0 lub inną wspieraną. Krótka instrukcja (https://achilles.tu.kielce.pl/portal/Members/332bd2d158a24964b2b98a7fc2879842/programowanie-wspolbiezne/laboratoria/net-faq-how-to/zmiana-wersji-net-core-projektu)

1. **Prosty delegat (1,0)**

```
delegate void PrintSomething(string sth);

static void PrintOryginal(string sth)
        {
            Console.WriteLine(sth);
        }

 static void PrintInCapital(string sth)
        {
            Console.WriteLine(sth.ToUpper());
        }

 static void Main(string[] args)
        {
            PrintSomething printingDelegate = new PrintSomething(PrintOryginal);
            printingDelegate("Hello World");
            printingDelegate = new PrintSomething(PrintInCapital);
            printingDelegate("Hello World");
            printingDelegate += new PrintSomething(PrintOryginal);
            printingDelegate("Hello New World!");
            Console.WriteLine("Printing delegate has following methods:");
            foreach (var item in printingDelegate.GetInvocationList())
            {
                Console.WriteLine(item.Method.Name);
            }
        }
```

   - Na podstawie kodu w projekcie *SimpleDelegate* zdefniować delegata typu *void* z dwoma parametrami typu *int*.
   - Do podanego delegata utworzyć 4 metody wypisujące sumę, różnicę, iloraz i iloczyn.
   - Dodać nowe metody z poprzedniego podpunktu do listy wywołań delegata.
   - Uruchomić delegata
   - Wypisać nazwy metod z listy wywołań delegata.

2. **Kończenie v1 (0,5)**
   - Uruchomić Projekt *SimpleEvent*.
   - Program nie kończy się nawet gdy naciśniemy dowolny klawisz.
     Powodem jest oczekiwanie na potomka.
   - W metodzie *startClock* ustaw właściwość wątku *IsBackground* na *true*.
   - Uruchomić ponownie. Nie czekając na uruchomienie eventa nacisnąć dowolny klawisz.

3. **Kończenie v2 (1,0)**
   - Zakomentuj dododaną linię
   - Niech klasa Alarm implementuje interfejs *IDisposable*
   - Tu głównie chodzi o metodę *Dispose,* w której Wykonamy operację Interrupt() na wątku internalTrhead po czym przypiszemy mu null;
   - Wątek powinien obsłużyć wyjątek *ThreadInterruptedException* powodując jego zakończenie ze stosownym komunikatem.
   - W *Main* zaraz po odczycie klawisza wywołaj alarm.Dispose();
   - Uruchom program. Sprawdź działanie.

4. **Możliwość podania *checkingInterval (0,5)***
   - Przeciążyć konstruktory i metody *SetAlarm* o możliwość podania *checkingInterval*.
   - Obecnie domyślnie jest 1000ms.
5. **Nowy Event (1,0)**
   - W checkingTime dodać aktualizację licznika pozostałego czasu *timespanToAlarm*,
   - Do klasy Alarm dodać własność tylko do odczytu zwracającą wartość *timespanToAlarm*
   - Dodać *eventa* wyzwalanego przy każdym sprawdzaniu (metoda checkingTime)
   - Wraz ze sprawdzaniem event powinien zwrócić pozostały czas do alarmu.
   - Zademonstrować działanie w programie. Czyli między innymi dodać obsługę nowego zdarzenia.

6. **TimerExampe (0,5)**
   - Uruchomić projekt *TimerExample*.
     Widzimy, że *timerThreading* uruchamiamy po pięciu sekundach w odstępach jednosekundowych. Natomiast *timerSystem* od razu w odstępach 1,5s
   - Dodaj linijkę
     `timerSystem.AutoReset = false;`
     To spowoduje że *timer* będzie działał jak jednorazowy budzik.
   - Dodajemy licznik. Do programu dodać poniższą metodę:

```
private static void TimerTask(object timerState)
    {
        Console.WriteLine($"{DateTime.Now:HH:mm:ss.fff}: starting a new callback.");
        var state = timerState as TimerState;
        T.Interlocked.Increment(ref state.Counter);
    }
```

   - Oraz klasę

```
    class TimerState
    {
        public int Counter;
    }
```

   - W *Main* dodaj obiekt klasy *`TimerState timerState = new TimerState { Counter = 0 };`*
   - *Podmień proste* wyświelanie *na wywołanie TimerTask*

```
T.Timer timerThreading = new T.Timer(new T.TimerCallback(TimerTask), timerState, 500
```

   - Uruchom

7. **Wyłączenie timera (0,5)**
   - Używając `timerThreading.Change(T.Timeout.Infinite, 0);`
     Wyłącz timer po 5 wywołaniach (gdy licznik osiągnie 4)

8. **Inne czasomierze.**
   Niektóre biblioteki mogą zawierać swoje czasomierze:
   - *System.Windows.Forms.Timer*: składnik Windows Forms, który w jednakowych odstępach czasu wyzwala zdarzenie. Składnik nie ma interfejsu użytkownika i jest przeznaczony do użytku w środowisku jednowątkowym.
   - *System.Web.UI.Timer*: składnik ASP.NET, który wykonuje asynchroniczne lub synchroniczne operacje zwrotne stron internetowych w regularnych odstępach czasu.
   - *System.Windows.Threading.DispatcherTimer*: czasomierz zintegrowany z kolejką, która jest przetwarzana w określonym interwale czasu i Dispatcher z określonym priorytetem.